

Arquitectura web — mendelEmbeddings

v0.1 (draft, ajustable) — sesión de diseño 2026-05-20

Diego + Claude

2026-05-20

Arquitectura web de mendelEmbeddings — v0.1

Estado: borrador inicial. Es la primera versión “viva” después de la sesión de diseño del 2026-05-20. Cuando se ajuste, se crea una `v0.2_<fecha>_arquitectura_web.md` al lado de este archivo. **No se edita retroactivamente** — las versiones son snapshots inmutables; los cambios viven en versiones nuevas.

TL;DR (1 párrafo)

El proyecto se monta como **servicio web público** con arquitectura **híbrida cliente/servidor**: inference facial corre en el browser del usuario (privacidad por diseño, latencia baja, costo de server bajo), y el servidor existe solo para “refinamiento avanzado” a pedido de usuarios registrados. Se desarrollan en paralelo **dos tracks**: (1) *vitrina equipos de fútbol*, completamente estática y curada por Diego, y (2) *comparador de rostros niño-padres*, que arranca 100% en cliente y se extiende con server-side opcional. Casi todo el desarrollo se puede hacer **en local** durante muchos meses antes de necesitar un host.

1. Cómo llegamos acá

1.1 Versión preliminar (mañana del 2026-05-20)

Recomendación inicial: stack tradicional “FastAPI server + SPA frontend separada + Postgres + Redis + ARQ + auth managed + Fly.io”. Plan **Fase A → B → C** secuencial: motor primero (Tarea #1, completada hoy), después app primaria niño-padres, después app secundaria equipos, después escala.

1.2 Ajuste de la tarde (este documento v0.1)

El usuario aportó **tres ajustes que reformaron el plan**:

1. **“Quiero correr los tests localmente hasta tener un MVP que desplegar.”** → Toda la arquitectura debe ser local-first, con `docker-compose` para los pocos servicios que requieran daemon.
2. **“Empezar por la vitrina equipos, no por niño-padres. Mientras tanto, ir desarrollando ambas en paralelo.”** → Las dos apps son ahora **tracks paralelos**, no fases secuenciales. La vitrina es estática y sin necesidad de auth/storage/queue → no requiere casi nada de la complejidad backend.
3. **“¿Hay posibilidad de correr modelos directamente en el cliente? Híbrida: visualización casi instantánea 100% privada en cliente + refinamiento opcional en server.”** → Cambia el diseño: el server pasa de ser el corazón del producto a ser una **opción premium para usuarios registrados**. Privacidad pasa de “polícita” a “diseño”.

Estos tres ajustes simplificaron 6 de las 8 decisiones abiertas (W1–W8) — la mayoría quedan diferidas a la fase de escala.

2. El plan en 2 tracks paralelos

Track 1 — Vitrina equipos de fútbol

Caso de uso: comparar selecciones / planteles, generar matrices de similitud, dendrogramas, boxplots intra-grupo. Datos curados exclusivamente por Diego.

Stack:

- Cliente Vite + React (SPA).
- Datos: un único JSON pre-calculado con embeddings de cada jugador (generado offline por un script Python que usa el motor `phyloface`).
- Hosting: estático (Cloudflare Pages / Netlify / GH Pages — sin decidir).
- **No requiere:** server, DB, auth, storage, cola async.

Run local: `cd client && npm run dev` + cuando agrega fotos: `python3 scripts/build_teams_embeddings.py`. Listo.

Cuándo salta a hosting: cuando se quiere compartir el URL con alguien.

Track 2 — Comparador de rostros niño-padres

Este track tiene **dos sub-tracks** que se construyen en orden:

Track 2a — Cliente “puro” (anónimo, sin server)

Inference facial **100% en el browser** vía:

- **MediaPipe Tasks for Web** (Google, JS oficial) — face detection + 468 landmarks. Equivalente directo al subpaquete `phyloface.landmarks`.
- **ONNX Runtime Web** — corre el mismo modelo `w600k_r50.onnx` que usa el motor Python, en el browser (WebGPU > WebGL > WASM como fallbacks).
- Comparación por regiones: portar `phyloface.regions` y `phyloface.comparator_regional` a JS (operaciones puras sobre arrays — son ~300 líneas, factibles).

Resultado para el usuario anónimo:

Feature	En cliente?	Latencia esperada
Score global (coseno entre dos embeddings)	✓	~10ms
Comparación visual simple por región (grayscale + coseno)	✓	~100ms
Embedding por región Nivel B (re-aplica modelo a 12 crops)	✓ con WebGPU	~2-3s
Visualizaciones (heatmap, radar, overlay de regiones)	✓	~10ms
Occlusion sensitivity / saliency maps	⚠ Posible pero lento (10-60s) → mejor server	—
Calibración con dataset de validación	✗ Server	—

Imágenes y datos personales: nunca salen del browser. Privacidad por diseño.

Track 2b — Server-side “refinamiento” (registrado opt-in)

Cuando el usuario registrado pide “comparación avanzada”:

- El cliente envía `{image, embeddings_precalculated, landmarks_precalculated}` al server.
- El server **valida** que sus embeddings coinciden con los del cliente (sanity check de integridad — detecta browsers con WebGPU buggy).
- El server **salta** detección + embedding global (ya hechos) y solo corre lo que el cliente no puede:
 - Embedding por región Nivel B (más preciso aplicando regiones canónicas calibradas).
 - Occlusion sensitivity completa (60s-2min de inferencias).
 - Calibración del score contra umbral data-driven (Tarea #6).

- Persistencia del historial.
- Resultado: refinamiento + score calibrado + heatmap explicativo.

Ahorro operativo concreto: como el cliente ya hizo detección + embedding global ($\approx 30\%$ del trabajo total), el server reduce su carga $\sim 30\%$ por request → más usuarios concurrentes con menos workers.

3. Las 8 decisiones (W1–W8) — estado actual

#	Decisión	Estado	Resolución
W1	Framework FE	✅ Cerrada	React + Vite SPA pura, sin SSR (no necesita SEO). TanStack Query para state-server cuando entre el Track 2b.
W2	Auth	" Diferida (Track 2b)	Cuando llegue: Clerk o Supabase Auth (managed) con OAuth Google + magic link. Free tier hasta ~10k users.
W3	Persistencia	" Diferida (Track 2b)	Cuando llegue: Postgres (Neon dev branch + prod) + Redis (Upstash) para cache de resultados y backend de cola ARQ. Mientras tanto, IndexedDB del browser para historial local del usuario anónimo.
W4	Procesamiento pesado	" Diferida (Track 2b)	Cuando llegue: ARQ (async-nativo, Redis-backed) + WebSocket o polling para devolver resultado. Track 2a es 100% sincrónico en cliente.
W5	Imágenes subidas	✅ Resuelta vía arquitectura	Track 2a: imágenes nunca salen del browser. Track 2b: opt-in explícito por consentimiento + retención máx 90 días + encryption at-rest + S3-compatible (R2/B2).
W6	Hosting / deploy	🟡 Parcial	Hosting estático (CF Pages / Netlify / GH Pages) para cliente — sin decidir entre las 3 opciones, no bloquea desarrollo local. Cuando entre Track 2b: Fly.io para server.
W7	Estructura repo	✅ Cerrada	Monorepo : <code>client/</code> , <code>scripts/</code> , <code>src/phyloface/</code> , <code>server/</code> (cuando exista).
W8	Modelos en memoria	" Diferida (Track 2b)	Cuando llegue: warm pool — una instancia del modelo por worker, cargada en startup. ~500MB RAM por worker, limita concurrencia a 4-8 según RAM.

Resumen: para los próximos meses solo necesitamos W1 (React+Vite) y W7 (estructura monorepo). Las otras 6 entran cuando entremos al Track 2b.

4. Stack concreto recomendado

Cliente (ambos tracks)

Capa	Tecnología	Justificación
Build / dev server	Vite	Más rápido que CRA/Next para SPA; HMR instantáneo
UI framework	React 18+	Más recursos, ecosystem maduro, mejor para reclutamiento futuro
Routing	React Router o TanStack Router	Track 1 (/teams) y Track 2 (/compare) viven en la misma SPA
State (server-side)	TanStack Query	Cache, retries, polling, mutations — perfecto para Track 2b
State (local UI)	Zustand o <code>useState/useReducer</code>	Simple, sin boilerplate
Estilos	Tailwind CSS	Velocidad de iteración, sin context-switch a CSS files
Viz	D3.js (heatmap, dendrograma) + Recharts (boxplot, radar)	D3 para lo custom, Recharts para lo standard
Inference (Track 2a)	ONNX Runtime Web + MediaPipe Tasks for Web	Mismo modelo que Python (<code>w600k_r50.onnx</code>), mismo backbone de landmarks

Pipeline offline (Track 1)

Componente	Tecnología
Script de generación	Python + paquete <code>phyloface</code> existente
Output	JSON con <code>{player_id, embedding[512], metadata}</code> por jugador
Trigger	Manual (Diego lo corre cuando agrega fotos)

Server (solo Track 2b, sin urgencia)

Capa	Tecnología	Cuándo
HTTP framework	FastAPI	Track 2b
Cola async	ARQ (Redis-backed)	Track 2b
ORM	SQLAlchemy + Alembic (migrations)	Track 2b
Auth	Clerk o Supabase (managed)	Track 2b
Object storage	Cloudflare R2 o Backblaze B2	Track 2b

Infraestructura local

Componente	Tecnología	Cuándo entra
Postgres + Redis	docker-compose	Track 2b — un solo <code>docker compose up -d</code> y listo
Tunnel para demos	Cloudflare Tunnel o ngrok	Cuando quiero mostrar el localhost a alguien sin deploy

5. Estructura propuesta del repo

```
mendelEmbeddings/
├── src/phyloface/                                # Motor Python – ya existe ✅ (Tarea #1)
│   ├── core/                                    # io, metrics, embedder, pairs, comparator_global
│   ├── landmarks/                              # MediaPipe Face Mesh
│   ├── regions/                                # geometry, extract_rect, extract_masked
│   ├── comparator_regional.py
│   ├── viz/                                    # detection, landmarks, regions, heatmap
│   └── app/                                     # apps existentes (run_pairwise_heatmap, etc.)
├── scripts/                                    # NUEVO – entry points CLI Python
│   ├── build_teams_embeddings.py                # Track 1: genera JSON de equipos
│   └── verify_onnx_web_parity.py                # Track 2a: spike – verifica que ONNX en JS da iguales
embeddings que Python
├── client/                                     # NUEVO – SPA con ambos tracks
│   ├── public/
│   │   ├── teams_embeddings.json                # Track 1 (copiado del output del script Python)
│   │   └── models/                             # Track 2a
│   │       ├── face_detector.tflite            # MediaPipe (opcional según método)
│   │       ├── face_landmarker.task            # MediaPipe FaceMesh
│   │       └── w600k_r50.onnx                  # InsightFace embedding
│   ├── src/
│   │   ├── apps/
│   │   │   ├── teams/                         # Track 1: rutas + componentes
│   │   │   └── compare/                       # Track 2: rutas + componentes
│   │   ├── lib/                               # Compartido
│   │   │   ├── inference/                     # Track 2a: wrapper de ONNX Runtime Web
│   │   │   ├── viz/                           # primitivas D3, paleta, types
│   │   │   └── api/                           # Track 2b: cliente HTTP del server
│   │   └── main.tsx
│   ├── package.json
│   └── vite.config.ts
├── server/                                    # NUEVO – Track 2b (no en MVP)
│   ├── app/
│   │   ├── routes/
│   │   ├── workers/                           # tareas ARQ
│   │   ├── models/                             # SQLAlchemy
│   │   └── main.py                             # FastAPI startup
│   ├── alembic/
│   └── pyproject.toml
├── docker-compose.yml                         # NUEVO – Postgres + Redis local (Track 2b)
├── _meta/
│   ├── MIGRACION_TAREA1.md                    # ✅ Cerrado
│   └── arquitectura_web/                       # NUEVO – este snapshot vive acá
│       └── v0.1_2026-05-20_arquitectura_web.md (+ .pdf)
├── data/
│   ├── input/                                 # imágenes curadas (equipos, ejemplos)
│   └── output/
```



```
| notebook_runs_*/ # outputs del notebook (gitignored)
|
|_toReview/ # archivo experimental ya archivado ✓
```

6. Privacidad / legal

6.1 Marco regulatorio

- **RGPD (UE)**: aplica si hay usuarios europeos. Las caras y los embeddings faciales son **datos biométricos** (categoría especial, art. 9) → consentimiento explícito + propósito específico + derecho al borrado + minimización.
- **Ley 25.326 (Argentina)**: similar. Caras = datos sensibles. Inscripción en el Registro Nacional de Bases de Datos si se guardan.

6.2 Cómo el diseño aborda esto

Caso	Tratamiento
Usuario anónimo (Track 2a)	Imágenes y embeddings nunca salen del browser . No hay base de datos a inscribir. No hay consentimiento porque no hay procesamiento de datos en el server.
Usuario registrado, comparación normal (Track 2a)	Igual que anónimo. El registro existe para acceder a Track 2b, no para registrar comparaciones normales.
Usuario registrado, refinamiento Track 2b	Consentimiento explícito al momento de pedir refinamiento (no enterrado en T&C generales). Imagen procesada y borrada salvo opt-in separado para historial. Embeddings persisten si el usuario opta-in. Retención máx 90 días por default. Right to be forgotten = DELETE simple.

6.3 Caso especial Track 1 (vitrina equipos)

Jugadores de fútbol profesional son **personas públicas** en contexto de su actividad → cierta latitud bajo “interés público” y “datos manifiestamente publicados por el titular”. Pero **igual** son datos biométricos. Mitigación:

- Solo procesamos fotos públicas oficiales (rosters, prensa).
- No se almacena la cara del jugador como imagen en el servidor; solo el embedding (que en este contexto está dentro del propósito declarado: comparación entre miembros del mismo grupo público).
- Política de privacidad explícita: “datos públicos, procesados para análisis estadístico de similitud, sin almacenamiento de imágenes en server”.

6.4 Acciones concretas pendientes

- Redactar Política de Privacidad simple (1 página).
- Redactar Términos de Servicio (1 página).
- Si se monta Track 2b en producción: inscripción en Registro Nacional de Bases de Datos (AR) — trámite simple, gratuito.

7. Cómo correr todo en local — hasta dónde llega

Componente	Local funciona?	Cuándo salir
Vitrina equipos (Track 1 completa)	✅ 100% local con <code>npm run dev</code> + script Python	Cuando quiero compartir el URL
Cliente comparador anónimo (Track 2a)	✅ 100% local	Idem
Server FastAPI + ARQ worker (Track 2b)	✅ 100% local con <code>docker compose up -d</code> (Postgres + Redis en contenedores)	Cuando voy a recibir tráfico real
Auth Clerk/Supabase	✅ 95% local — dev tier permite callbacks a <code>localhost:5173</code>	Cuando quiero OAuth con dominios reales
Persistencia managed (Neon)	✅ 100% local — apuntar localhost al Neon dev branch	Cuando entro en deploy continuo
Mostrar a alguien sin deploy	🟡 Posible con ngrok / Cloudflare Tunnel (URL pública temporal)	Cuando ngrok no alcanza
Validar concurrencia real (varios usuarios simultáneos)	❌ Requiere host	Cuando hay tráfico beta
Validar performance de descarga del modelo desde CDN	❌ Requiere host	Idem

Estimación práctica: 6–12 meses calendario de desarrollo 100% local hasta querer mostrar a usuarios beta. Cuando ese día llegue, deploy a CF Pages (cliente) + Fly.io (server) = 1-2 días de setup.

8. Telemetría / “rescatar info” del cliente

Usuario	Qué se envía al server	Justificación
Anónimo	Nada por default	Sin consentimiento, no hay procesamiento de datos en el server. Si se quisiera telemetría agregada (ej. “distribución de scores globales”, sin embeddings individuales), debe ser opt-in claro y agregado (no per-evento).
Registrado, comparación normal Track 2a	Nada	Igual que anónimo. El registro habilita Track 2b, no implica telemetría automática.
Registrado, refinamiento Track 2b	{image, embeddings_precalculated, landmarks_precalculated}	Consentimiento explícito al pedir refinamiento. Imagen procesada y descartada salvo opt-in separado para historial.

Nota importante: los embeddings faciales son **datos biométricos sensibles** bajo RGPD y ley 25.326 AR — tan o más sensibles que las imágenes (identificadores únicos pseudo-irreversibles). El default conservador es no recolectar.

9. Riesgos identificados / cosas a validar

9.1 Riesgos técnicos

Riesgo	Mitigación / spike
ONNX Runtime Web no carga w600k_r50.onnx o da embeddings diferentes que Python	Spike Track 2a inicial (scripts/verify_onnx_web_parity.py + test JS con la misma imagen). Si falla, todo el plan híbrido se cae → habría que volver al plan original con server-only inference.
Latencia del modelo en cliente sin WebGPU (Safari, Firefox)	Fallback a server-side para esos browsers, o aceptar 10x más lento en WASM.
50MB de modelo a descargar primera vez = experiencia inicial lenta	Service Worker para cache + UI claro (“preparando modelo, primera vez tarda ~30s”).
RAM del browser con modelo cargado	Validar en spike — probablemente OK con WebGPU/WebGL backends.

9.2 Riesgos de producto

Riesgo	Mitigación
El JSON de equipos crece y se vuelve pesado al cargar	Particionar por equipo / país; lazy load por sección de la vitrina.
Embeddings exportados a JS divergen del motor Python con el tiempo	Test de paridad en CI cuando exista (compara embeddings JS vs Py sobre fixture).
RGPD: usuario quiere borrado de su historial	Track 2b debe tener endpoint DELETE explícito desde día 1.

9.3 Riesgos de proceso

Riesgo	Mitigación
Avanzar en paralelo en dos tracks sin coordinación → divergencia de código compartido	Mantener <code>client/src/lib/</code> como zona común con tests; nada de copy-paste entre <code>apps/teams/</code> y <code>apps/compare/</code> .
Esta versión v0.1 se "olvida" cuando empezamos a tirar código y se ajustan cosas	Cuando se ajuste, crear <code>v0.2_<fecha>_arquitectura_web.md</code> (no editar v0.1). Releer este doc al inicio de cada sesión web.

10. Próximos pasos inmediatos

Próxima sesión (decidido): **Track 2a, spike de viabilidad ONNX Runtime Web.**

Plan de la sesión:

1. Crear `client/` con Vite + React + TypeScript (setup mínimo).
2. Bajar `w600k_r50.onnx` del modelo InsightFace (`buffalo_l/w600k_r50.onnx` , ~92MB).
3. Crear `scripts/verify_onnx_web_parity.py` : corre el motor Python sobre una cara de prueba, guarda el embedding como JSON reference.
4. En el cliente: cargar el ONNX con `onnxruntime-web` , pasar la misma cara (subida o de `public/`), obtener embedding, comparar con el reference JSON.
5. Criterio de éxito: `cosine_similarity(emb_py, emb_js) > 0.9999` y `max(|diff|) < 1e-3` (cuasi-bit-identical aceptable; diferencias $< 1e-5$ si la cuantización del modelo es FP32 puro).

Si el spike funciona → seguimos con Track 2a real (alineación, regiones, viz). **Si el spike falla** → volvemos a v0.2 con plan revisado (probable: server-side inference + cliente solo viz).

11. Apéndice — Glosario

- **SPA** (Single Page Application): app web cargada una sola vez; routing y datos via JS, sin recargas de página.
- **SSR** (Server-Side Rendering): cada página se renderiza en el server (Next.js, Nuxt). Bueno para SEO y carga inicial; no necesario acá.
- **ONNX** (Open Neural Network Exchange): formato standard para modelos de ML. Permite correr el mismo modelo en Python, JS, C++, mobile.
- **ONNX Runtime Web (onnxruntime-web)**: librería JS para correr modelos ONNX en browser con backends WebGPU / WebGL / WASM.
- **WebGPU**: API moderna del browser para acceso a GPU (Chrome , Safari 17+ , Firefox parcial). Necesaria para inference rápida.
- **WASM** (WebAssembly): binary format que corre en browser a ~80% velocidad nativa. Fallback de ONNX Runtime Web cuando no hay GPU.
- **ARQ**: librería Python de cola de tareas async (basada en Redis), alternativa moderna y minimal a Celery.
- **Spike**: un experimento corto y acotado para validar viabilidad de algo antes de invertir en construirlo completo.
- **MediaPipe Tasks for Web**: SDK de Google para correr modelos de visión (face detection, landmarks, hand tracking) en browser.
- **TanStack Query** (ex React Query): librería de state-server para frontend; cache, retries, polling.
- **TLS / HTTPS local**: requerido por WebRTC, getUserMedia (cámara), service workers. Vite lo provee con `--https` flag.
- **Magic link**: link de un solo uso enviado por email para login sin password.

12. Historial de cambios

Versión	Fecha	Autor	Cambios
v0.1	2026-05-20	Diego + Claude	Versión inicial. Define plan en 2 tracks paralelos con inference híbrida cliente/server. Decide stack de Fase A (cliente). Difiere 6 de 8 decisiones W al Track 2b.